



ISLAMIC UNIVERSITY OF TECHNOLOGY
ORGANISATION OF ISLAMIC COOPERATION (OIC)
DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING



EEE 4706 Project Report

Project Title: DC Motor with Speed Control

Lab Group: B2

Project Group: 03

Group Members Information:

Name	Student ID
1. Ishrak Anan	210021212
2. Iffat Jahan	210021222
3. Md. Sabbir Hossain Tasrif	210021234
4. Seam Saif	210021252
5. Lamin Touray	210021260
6. Enayatullah Zurmati	210021246

1. Objective

The objective of this project is to design and implement a keypad-controlled DC motor control system using an 8051-based microcontroller trainer/development board, a motor driver interface, and a 16×2 LCD. The system is intended to control motor direction and speed through keypad input while displaying the operating status on the LCD. The project integrates software-based PWM control, safe direction-changing logic, and protection features into a complete embedded motor-control application demonstrated through both simulation and hardware implementation. The specific objectives are to:

- control motor rotation in clockwise and anti-clockwise directions through keypad commands and display the selected status on the LCD;
- accept exact speed input from the keypad and apply the corresponding PWM duty cycle for motor speed control;
- implement safe reversal logic to avoid sudden direction switching during operation;
- include protection logic that detects continuous 100% duty operation for a defined period and forces the system into a safer reduced-speed mode;
- provide additional monitoring and indication features such as acceleration-time display, LED-based speed indication, and fault-status indication.

2. Required Components

Component	Qty.	Purpose	Estimated Unit Cost (BDT)	Subtotal (BDT)
8051 Microcontroller Development/Trainer Board	1	Main controller platform	7,500 Tk	
L293D motor driver IC	1	H-bridge motor interface	80 Tk	
16×2 LCD (LM016L / equivalent)	1	Status display	340 Tk	
DC motor	1	Actuator	145 Tk	
LEDs	3	Speed indication	15 Tk	
Breadboard, jump wires, connectors, Power supply	1 set	Prototype integration	300 Tk	
Estimated Total Cost				= 8,380 Tk

Figure 1. Final Proteus schematic of the DC motor controller.

The connection summary is listed below.

Signal Group	AT89C51 Pins	Connection / Function
LCD data bus	P2.0 – P2.7	Connected to D0 – D7 of the 16×2 LCD
LCD control	P0.0, P0.2	RS and E respectively; LCD RW tied to ground
Keypad rows/columns	P1.0 – P1.7	4×4 matrix keypad scanning
Motor direction	P3.0, P3.1	L293D IN1 and IN2
PWM output	P3.7	L293D EN1; enables software PWM speed control
LED indicators	P3.2, P3.3, P3.4	Low, medium, and high duty LEDs
Clock network	XTAL1, XTAL2	11.0592 MHz crystal with 22 pF capacitors
Reset / bias	RST and EA-related network	Push-button reset and required pull network

4. Features

4.1 Mandatory Features

4.1.1 Keypad Direction Control with Soft Reverse

This feature allows the user to control the direction of rotation of the DC motor directly from the keypad. In the project, the ‘+’ key is used to command clockwise (CW) rotation, the ‘-’ key is used to command counter-clockwise (CCW) rotation, and the ‘÷’ key is used to stop the motor. Whenever one of these commands is entered, the microcontroller updates the motor-control signals through the L293D motor driver and at the same time shows the current motor status on the 16×2 LCD.

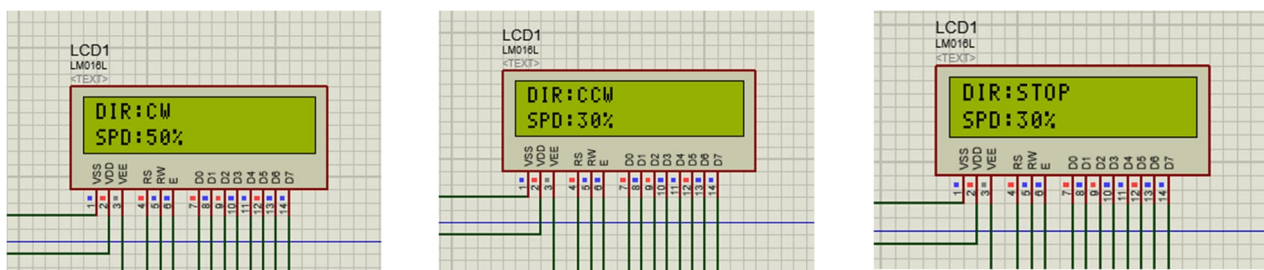


Figure 2: LCD Display Showing Direction Control States (CW, CCW, STOP) with Speed Indication

As shown in Figure 2, the display provides immediate visual confirmation of the selected operating state, such as DIR: CW, DIR: CCW, or DIR: STOP, together with the currently applied speed percentage.

A major part of this feature is the **soft reverse mechanism**. In a normal DC motor drive, instantly changing the direction while the motor is already running can produce a sudden reverse current, mechanical shock and unnecessary stress on both the motor and the driver circuit. To avoid this, the controller does not allow an immediate reversal. If the motor is rotating in one direction and the user requests the opposite direction, the firmware first stops the motor, then waits until the pressed key is released, applies a short software delay and only after that enables the new direction. This delay is approximately 500ms after key release. This makes the reversal smoother and safer.

Therefore, this feature serves two purposes at the same time: it provides easy user control of motor direction through the keypad, and it also introduces a protective control strategy for reversal. The LCD indication makes the operation clear to the user, while the soft-reverse sequence improves the reliability of the system by preventing abrupt switching between CW and CCW motion.

4.1.2 Exact Speed Entry by Keypad

This feature enables the user to set the motor speed directly from the keypad by entering a numeric value that represents the desired PWM duty cycle. The controller accepts the entered digits as the target speed command, and when the '=' key is pressed, the value is applied and displayed on the LCD as a percentage. In normal operation, the keypad supports the standard two-digit range from 00% to 99%, allowing simple and accurate speed selection.

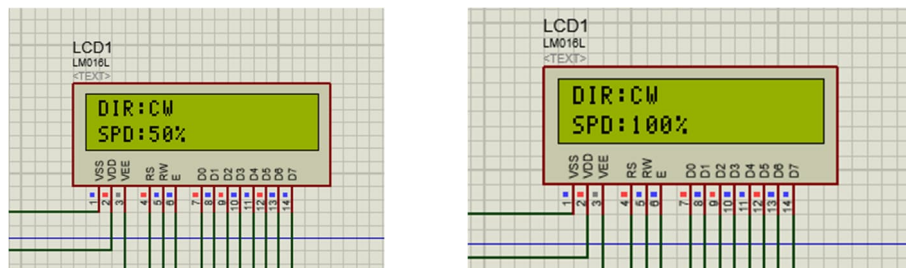


Figure 3: Exact Speed Entry by Keypad Demonstration Showing Direct PWM Duty-Cycle Input and LCD Display of Selected Speed Values (50% and 100%)

To extend the usable range, the firmware also includes a special three-digit handling path for the exact value 100. This makes it possible to command 100% duty cycle, which is especially important for testing full-speed operation and the thermal cutoff feature. Therefore, this feature provides a practical and user-friendly method of digital speed input while ensuring that the controller can cover the full speed range required by the project.

4.1.3 Thermal Cutoff Protection

A Timer0-driven background counter continuously monitors whether the motor is running at 100% duty cycle for 10 seconds without interruption. If this condition is reached, the controller assumes an overheat risk, sets an overheat flag, automatically lowers the duty cycle to a safer 50%, and displays a warning message on the LCD.

Once this protection is triggered, normal operation is not restored automatically. The system remains in the protected state until the user presses the C key, which clears the overheat latch and allows the motor to resume normal operation. In this way, the feature simulates a thermal cutoff protection system and demonstrates how an embedded controller can enforce safe operating limits even without using an actual temperature sensor.

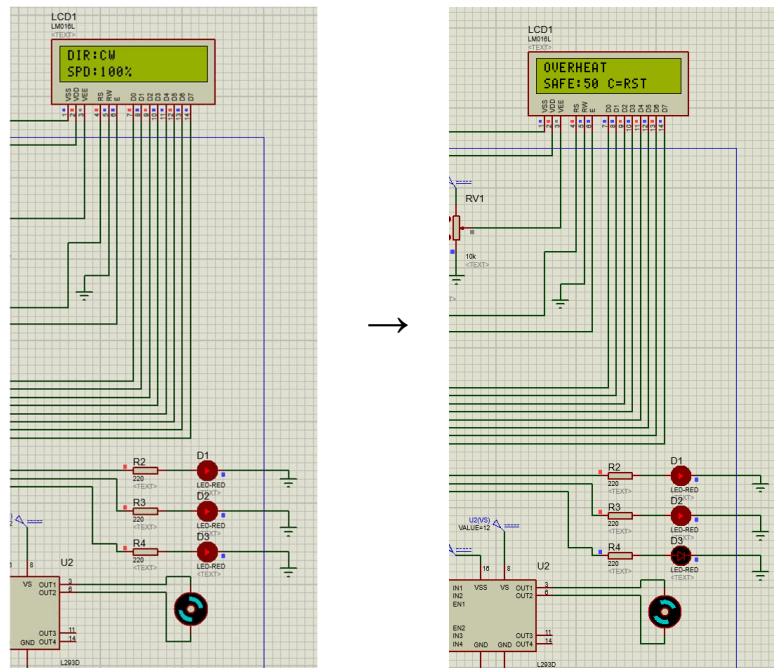


Figure 4: Thermal Cutoff Protection Demonstration Showing Overheat Detection and Automatic Duty Reduction

4.2 Proposed Additional Features

4.2.1 Motor Acceleration Time Monitoring

Motor acceleration time monitoring is used to measure how long the motor takes to reach a newly commanded speed during operation. When the motor is already running and the user enters a different speed from the keypad, the controller does not change the speed abruptly. Instead, it gradually adjusts the applied PWM duty cycle toward the new target in controlled steps. This allows the system to create a smoother transition between speed levels and at the same time measure the time required for that transition.

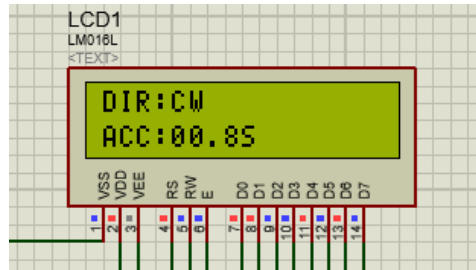


Figure 5: LCD Display Showing Motor Acceleration Time (ACC) During Speed Ramp

The measured value is temporarily shown on the LCD as the motor's acceleration time. Therefore, in this project, the feature serves as both a monitoring function and an improvement to speed control behavior. It demonstrates that the controller can handle speed changes in a controlled manner rather than applying sudden jumps, while also giving the user visible feedback about the time needed for the motor to reach the new operating condition.

4.2.2 Multi-Level LED Indication and Simulated Fault Detection

Three LEDs provide a quick visual estimate of the speed range. Duty 1–25% turns on LED1, 26–50% turns on LED1 and LED2, 51–99% turns on all three LEDs, and 100% causes all three LEDs to blink together. Because the current Proteus simulation does not include a real feedback path to sense open-circuit LEDs, fault detection is implemented as a controlled simulation: the × key toggles a demo fault state, all LEDs are suppressed, and the LCD displays an LED fault message. This makes the feature demonstrable within the project constraints.

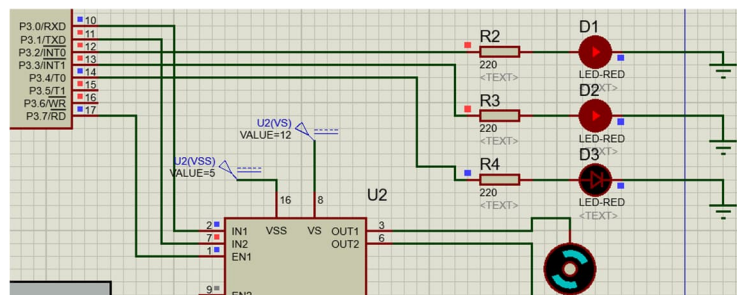


Figure 6: Multi-Level LED Speed Indication

4.3 Feature Verification

Feature	Test Input / Condition	Observed Result	Status
CW command	+	LCD shows DIR: CW and motor rotates clockwise	Verified
CCW command	-	LCD shows DIR: CCW and motor rotates anti-clockwise	Verified
Soft reverse	20 = + then -	Motor stops briefly, then reverses to CCW at same speed	Verified
Exact full speed	1 0 0 =	LCD shows SPD: 100%	Verified
Thermal cutoff	Run continuously at 100% for 10 s	Forced to 50% and OVERHEAT message shown	Verified
Manual reset	Press C during OVERHEAT	Overheat latch clears	Verified
LED indication	25 / 50 / 75 / 100	1 / 2 / 3 LEDs and blink 3 LEDs at 100%	Verified
LED fault demo	×	LED FAULT message shown	Verified
Acceleration display	While running, change 50% to 70%	ACC: xx.xS shown temporarily	Verified
Stop the motor	÷	Motor stops safely	Verified

5. Working Principle

The project operates as a keypad-controlled DC motor drive system built around the AT89C51 microcontroller. Its main purpose is to let the user control the direction, speed, and safety behavior of a DC motor through a simple human interface consisting of a 4×4 keypad, a 16×2 LCD, and visual LED indicators. The microcontroller acts as the brain of the system, continuously receiving user commands, processing them in software, and generating the proper control signals for the motor through the L293D motor driver IC.

Flowchart of Working Principle

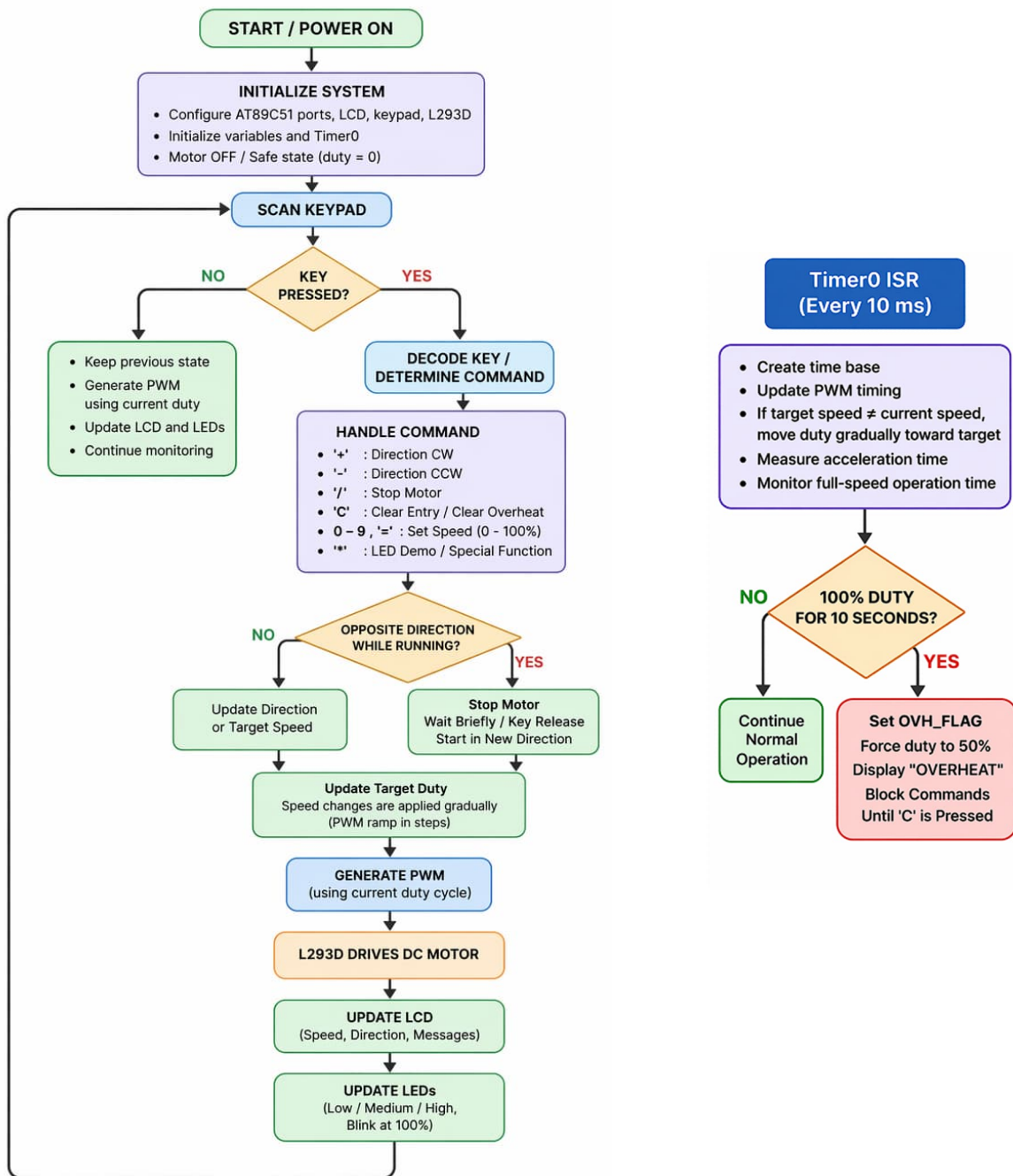


Figure 7. Algorithm flow of the final firmware.

At the beginning of operation, the microcontroller performs an initialization routine. During this stage, the input and output ports are configured, the LCD is initialized, the timer is started, and all software variables are set to their default values. The motor outputs are also kept in a safe inactive state so that the system starts without unintended motion. Once initialization is complete, the controller enters its main operating loop and remains there throughout the execution of the program.

The **keypad** is the primary user-input device. Through it, the user can enter direction commands, speed digits, stop commands, clear/reset commands, and special feature commands. Whenever a key is pressed, the microcontroller scans the keypad matrix, identifies the pressed key, and interprets it according to the program logic. Numeric keys are used to form the desired speed value, while dedicated keys are used for motor rotation control such as clockwise, counter-clockwise, stop, and system reset related actions. The entered information is then stored in software variables and used to update motor behavior.

The **LCD** provides real-time feedback to the user. It displays the present operating state of the motor such as direction, speed percentage, acceleration-time information, and warning messages. This makes the system interactive and easy to monitor. Instead of the motor operating silently in the background, the LCD ensures that every important action taken by the controller is visible to the operator.

The **L293D motor driver** serves as the interface between the low-power microcontroller and the higher-current DC motor. The AT89C51 cannot directly drive the motor because its pins cannot supply the required current and also need electrical isolation from inductive motor effects. Therefore, the microcontroller sends logic signals to the input pins of the L293D, and the driver delivers the required output to the motor. Two control lines determine the **direction of rotation**, while the **enable pin** is used for speed control.

The speed of the motor is controlled using **software PWM (Pulse Width Modulation)**. In this method, the motor is not driven by a constant analog voltage; instead, the controller rapidly switches the motor supply on and off. By adjusting the ratio of ON time to OFF time, called the **duty cycle**, the average voltage across the motor changes. A higher duty cycle gives a higher effective voltage and therefore higher speed, while a lower duty cycle produces lower speed. In this project, the duty cycle is entered by the user from the keypad and then applied through the software PWM routine. As a result, the motor speed becomes digitally selectable in percentage form.

A very important part of the system is the separation between the **foreground tasks** and the **background timing tasks**. The foreground loop handles user-dependent operations such as keypad scanning, LCD refreshing, command processing, and PWM execution. At the same time, **Timer0 interrupt service** runs periodically in the background and provides the time base for advanced functions such as acceleration timing, duty ramping, LED blinking, and overheat protection. This arrangement makes the controller responsive while still maintaining accurate timing behavior.

For safe operation, the project includes **soft reverse control**. If the motor is already rotating in one direction and the user requests the opposite direction, the system does not reverse immediately. Instead, it first stops the motor, waits briefly, and only then applies the new direction. This prevents sudden reverse current, reduces mechanical shock, and protects both the motor and the driver circuit from stress. In practical terms, this gives smoother and safer direction switching.

The system also supports **exact speed entry**. The user can enter a desired duty cycle through keypad digits, and after confirmation the value is stored as the target speed. If the motor is starting from rest, the selected speed is applied directly. If the motor is already running and a new speed is requested, the controller does not always jump abruptly to the new value. Instead, it can

gradually move the duty cycle toward the new target in controlled steps. This makes the response more realistic and allows timing-based features to be measured properly.

One of the proposed smart features of the project is **acceleration-time monitoring**. When the motor is already running and the user changes the speed command, the controller starts a timing count. Then it gradually changes the applied duty cycle until it reaches the new target. Once the target is reached, the elapsed time is stored and briefly shown on the LCD as the motor's acceleration or target-reaching time. This feature adds an analytical aspect to the controller and makes the project more than just a basic drive system.

Another important protection feature is the **thermal cutoff simulation**. If the motor is commanded to operate continuously at **100% duty cycle** for a defined period, the program assumes that the system has entered a risky overheating condition. It then activates an overheat flag, reduces the duty cycle to a safe level, and shows an OVERHEAT warning on the LCD. Normal commands are blocked until the user clears the condition manually. Although this is a software-based protection mechanism rather than a real temperature sensor system, it demonstrates how embedded controllers can enforce safe operating limits.

The project also uses **three LEDs** as visual speed indicators. Depending on the duty cycle, different LEDs turn on to show low, medium, or high speed ranges. At full speed, all LEDs can blink to represent maximum operating condition. This gives a quick visual understanding of the motor state even without reading the LCD. A simulated LED fault mode is also included so that fault handling can be demonstrated within the software.

Overall, the working principle of the project is based on the interaction of user input, real-time microcontroller processing, PWM-based motor driving, and software protection logic. The keypad provides commands, the AT89C51 interprets them, the L293D applies the proper motor drive signals, the LCD and LEDs show system status, and Timer0 maintains all time-dependent features in the background. In this way, the project demonstrates a complete embedded motor-control system that is functional, interactive, and safety-aware.

6. Code

```
LCD_RS      BIT P0.0
LCD_E       BIT P0.2
LED1        BIT P3.2
LED2        BIT P3.3
LED3        BIT P3.4

;----- RAM -----
DUTY        DATA 30H      ; applied duty 0..100
DIR          DATA 31H      ; 0=STOP,1=CW,2=CCW
KEYBUF       DATA 32H      ; last read key
D1_ASC       DATA 33H      ; first digit ASCII or 0FFh
D2_ASC       DATA 34H      ; second digit ASCII or 0FFh
D3_ASC       DATA 35H      ; third digit ASCII or 0FFh (only for 100)
OVH_FLAG     DATA 36H      ; 1=overheat latched
STATUS_REQ   DATA 37H      ; 1=refresh LCD
RUN_STATE    DATA 38H      ; 0=stopped,1=running
ACC_FLAG     DATA 39H      ; 1=elapsed-time timer active
ACC_DONE     DATA 3AH      ; 1=max-speed timing already captured for this run
ACC_TICKS    DATA 3BH      ; elapsed 100 ms ticks since startup
FULL100      DATA 3CH      ; continuous 100% runtime in 100 ms ticks
TICK10       DATA 3DH      ; 10 ms subcounter (0..9)
SHOW_ACC     DATA 3EH      ; 100 ms countdown for timing message display
BLINK_DIV    DATA 3FH      ; 100 ms divider for blink
BLINK_TGL    DATA 40H      ; 0/1 blink phase for 100% LEDs
LED_FAULT_S   DATA 41H      ; simulated LED fault (toggle with *)
LED_FAULT_F   DATA 42H      ; display flag for LED fault
TGT_TICKS    DATA 43H      ; elapsed time to desired-speed command
MAX_TICKS    DATA 44H      ; elapsed time to first 100% command
SHOW_MODE    DATA 45H      ; 0=none, 1=ACC
TARGET_DUTY   DATA 46H      ; selected/desired duty 0..100
RAMP_FLAG    DATA 47H      ; 1=software acceleration ramp active
```

```
ORG 0000H
LJMP START

ORG 000BH                      ; Timer0 interrupt vector
LJMP T0_ISR

;=====
; STARTUP
;=====
START:
    MOV SP, #6FH

    MOV P1, #0FFH
    MOV P0, #0FFH
    MOV P2, #00H

    CLR P3.0
    CLR P3.1
    CLR P3.7
    CLR LED1
    CLR LED2
    CLR LED3
```

```

MOV DUTY, #00
MOV DIR, #00
MOV KEYBUF, #00
MOV D1_ASC, #0FFH
MOV D2_ASC, #0FFH
MOV D3_ASC, #0FFH
MOV OVH_FLAG, #00
MOV STATUS_REQ, #01
MOV RUN_STATE, #00
MOV ACC_FLAG, #00
MOV ACC_DONE, #00
MOV ACC_TICKS, #00
MOV FULL100, #00
MOV TICK10, #00
MOV SHOW_ACC, #00
MOV BLINK_DIV, #00
MOV BLINK_TGL, #00
MOV LED_FAULT_S, #00
MOV LED_FAULT_F, #00
MOV TGT_TICKS, #00
MOV MAX_TICKS, #00
MOV SHOW_MODE, #00
MOV TARGET_DUTY, #50
MOV RAMP_FLAG, #00

; Timer0 mode1, 10 ms interrupt @ 11.0592 MHz
MOV TMOD, #01H
MOV TH0, #0DCH
MOV TL0, #00H
SETB ET0
SETB EA
SETB TR0

ACALL LCD_PWRDELAY
ACALL LCD_INIT
ACALL LCD_CLEAR
ACALL SHOW_STATUS

```

```

MAIN_LOOP:
    ACALL UPDATE_LEDS
    ACALL SERVICE_LCD

    ; ---- Read key ----
    ACALL GETKEY          ; A=ASCII or 00
    JNZ KEY_PRESENT
    LJMP DO_PWM

KEY_PRESENT:
    MOV KEYBUF, A

    ; If overheat is latched, only C can reset it
    MOV A, OVH_FLAG
    JZ  NORMAL_KEYS

    MOV A, KEYBUF
    CJNE A, #'C', WAIT_RELEASE
    ACALL RESET_OVERHEAT
    LJMP WAIT_RELEASE

```

NORMAL_KEYS:

```
; '+' => CW / soft reverse if needed
MOV A, KEYBUF
CJNE A, #'+', CHK_MINUS
ACALL CMD_CW
LJMP WAIT_RELEASE
```

CHK_MINUS:

```
MOV A, KEYBUF
CJNE A, #'-', CHK_STOP
ACALL CMD_CCW
LJMP WAIT_RELEASE
```

CHK_STOP:

```
MOV A, KEYBUF
CJNE A, #'/', CHK_CLR
ACALL STOP_MOTOR
ACALL UPDATE_RUN_AND_ACCEL
MOV STATUS_REQ, #01
LJMP WAIT_RELEASE
```

CHK_CLR:

```
MOV A, KEYBUF
CJNE A, #'C', CHK_EQ
MOV D1_ASC, #0FFH
MOV D2_ASC, #0FFH
MOV D3_ASC, #0FFH
MOV STATUS_REQ, #01
LJMP WAIT_RELEASE
```

CHK_EQ:

```
MOV A, KEYBUF
CJNE A, #'=', CHK_STAR
ACALL APPLY_SPEED
LJMP WAIT_RELEASE
```

CHK_STAR:

```
MOV A, KEYBUF
CJNE A, #'*', CHK_DIGIT
MOV A, LED_FAULT_S
CPL A
ANL A, #01H
MOV LED_FAULT_S, A
MOV STATUS_REQ, #01
LJMP WAIT_RELEASE
```

CHK_DIGIT:

```
; accept only '0'..'9'
MOV A, KEYBUF
CLR C
SUBB A, #'0'
JC WAIT_RELEASE

MOV A, KEYBUF
CLR C
SUBB A, #'9'+1
JNC WAIT_RELEASE

ACALL STORE_DIGIT
```

```
MOV STATUS_REQ, #01
LJMP WAIT_RELEASE
```

WAIT_RELEASE:

WR1:

```
ACALL GETKEY
JZ WR_DONE
; keep system alive while key is still held/repeating in Proteus
ACALL UPDATE_LEDS
ACALL SERVICE_LCD
ACALL PWM_ONE_PERIOD
SJMP WR1
```

WR_DONE:

```
LJMP DO_PWM
```

DO_PWM:

```
ACALL PWM_ONE_PERIOD
LJMP MAIN_LOOP
```

```
;=====
; TIMER0 ISR - 10 ms base
;=====
```

T0_ISR:

```
PUSH ACC
PUSH PSW
MOV PSW, #08H           ; register bank 1 used inside ISR so R4/R5/etc stay safe

MOV TH0, #0DCH
MOV TL0, #00H

INC TICK10
MOV A, TICK10
CJNE A, #10, T0_ISR_RETCHK
MOV TICK10, #00
SJMP T0_ISR_100MS
```

T0_ISR_RETCHK:

```
LJMP T0_EXIT
```

T0_ISR_100MS:

```
; 100 ms tasks -----

; show acceleration result only for a limited time
MOV A, SHOW_ACC
JZ T0_ACC_CNT
DEC SHOW_ACC
MOV A, SHOW_ACC
JNZ T0_ACC_CNT
MOV STATUS_REQ, #01
```

T0_ACC_CNT:

```
; acceleration/ramp timer increments while a commanded speed change
; is driving DUTY toward TARGET_DUTY
MOV A, ACC_FLAG
JZ T0_BLINK
INC ACC_TICKS

; software ramp: move DUTY by 5% every 100 ms until target is reached
```

```
MOV A,DUTY
CJNE A,TARGET_DUTY,T0_RCMP
SJMP T0_RDONE
```

T0_RCMP:

```
JC T0_RUP

; DUTY > TARGET_DUTY => ramp down
MOV A,DUTY
CLR C
SUBB A,#5
MOV R4,A
MOV A,TARGET_DUTY
CLR C
SUBB A,R4
JC T0_RSETDN
MOV A,TARGET_DUTY
MOV DUTY,A
SJMP T0_RDONE
```

T0_RSETDN:

```
MOV A,R4
MOV DUTY,A
SJMP T0_BLINK
```

T0_RUP:

```
; DUTY < TARGET_DUTY => ramp up
MOV A,DUTY
ADD A,#5
MOV R4,A
MOV A,R4
CLR C
SUBB A,TARGET_DUTY
JC T0_RSTORE
MOV A,TARGET_DUTY
MOV DUTY,A
SJMP T0_RDONE
```

T0_RSTORE:

```
MOV A,R4
MOV DUTY,A
SJMP T0_BLINK
```

T0_RDONE:

```
MOV ACC_FLAG,#00
MOV RAMP_FLAG,#00
MOV A,ACC_TICKS
MOV TGT_TICKS,A
MOV SHOW_MODE,#01
MOV SHOW_ACC,#20
MOV STATUS_REQ,#01
```

T0_BLINK:

```
INC BLINK_DIV
MOV A,BLINK_DIV
CJNE A,#5,T0_OVCHK ; toggle every 500 ms
MOV BLINK_DIV,#00
MOV A,BLINK_TGL
CPL A
```

```
ANL A, #01H
MOV BLINK_TGL, A
```

T0_OVCHK:

```
; if running continuously at 100%, count to 10 s
MOV A, OVH_FLAG
JNZ T0_EXIT          ; already latched
```

```
MOV A, DIR
JZ  T0_CLR100
```

```
MOV A, DUTY
CJNE A, #100, T0_CLR100
```

```
INC FULL100
MOV A, FULL100
CJNE A, #100, T0_EXIT
```

```
; OVERHEAT condition reached
MOV OVH_FLAG, #01
MOV DUTY, #50
MOV TARGET_DUTY, #50
MOV FULL100, #00
MOV ACC_FLAG, #00
MOV RAMP_FLAG, #00
MOV STATUS_REQ, #01
SJMP T0_EXIT
```

T0_CLR100:

```
MOV FULL100, #00
```

T0_EXIT:

```
POP PSW
POP ACC
RETI
```

```
=====
; COMMAND ROUTINES
=====
```

CMD_CW:

```
; opposite direction while running => soft reverse
MOV A, DIR
CJNE A, #02, CW_DIRECT
MOV A, RUN_STATE
JZ  CW_DIRECT
ACALL SOFT_REV_TO_CW
RET
```

CW_DIRECT:

```
ACALL SET_CW
ACALL UPDATE_RUN_AND_ACCEL
MOV STATUS_REQ, #01
RET
```

CMD_CCW:

```
MOV A, DIR
CJNE A, #01, CCW_DIRECT
MOV A, RUN_STATE
JZ  CCW_DIRECT
ACALL SOFT_REV_TO_CCW
```

```

    RET
CCW_DIRECT:
    ACALL SET_CCW
    ACALL UPDATE_RUN_AND_ACCEL
    MOV STATUS_REQ, #01
    RET

SOFT_REV_TO_CW:
    ACALL STOP_MOTOR
    ACALL UPDATE_RUN_AND_ACCEL
    MOV STATUS_REQ, #01
    ACALL SERVICE_LCD
SRCW_REL:
    ACALL GETKEY
    JNZ SRCW_REL
    ACALL SOFT_DELAY
    ACALL SET_CW
    ACALL UPDATE_RUN_AND_ACCEL
    MOV STATUS_REQ, #01
    RET

SOFT_REV_TO_CCW:
    ACALL STOP_MOTOR
    ACALL UPDATE_RUN_AND_ACCEL
    MOV STATUS_REQ, #01
    ACALL SERVICE_LCD
SRCCW_REL:
    ACALL GETKEY
    JNZ SRCCW_REL
    ACALL SOFT_DELAY
    ACALL SET_CCW
    ACALL UPDATE_RUN_AND_ACCEL
    MOV STATUS_REQ, #01
    RET

SOFT_DELAY:
    MOV R4, #250                ; short visible pause before reverse
SD1:
    ACALL DLY_1MS
    ACALL DLY_1MS
    DJNZ R4, SD1
    RET

RESET_OVERHEAT:
    MOV OVH_FLAG, #00
    MOV FULL100, #00
    MOV STATUS_REQ, #01
    RET

;=====
; Direction helpers
;=====
SET_CW:
    MOV DIR, #01
    SETB P3.0
    CLR  P3.1
    RET

```

```

SET_CCW:

```

```

MOV DIR, #02
CLR  P3.0
SETB P3.1
RET

```

STOP_MOTOR:

```

MOV DIR, #00
CLR  P3.0
CLR  P3.1
CLR  P3.7
MOV FULL100, #00
RET

```

```

;=====
; APPLY SPEED
;=====

```

APPLY_SPEED:

```

MOV A, D1_ASC
CJNE A, #0FFH, AP1
RET
; nothing entered

```

AP1:

```

MOV A, D3_ASC
CJNE A, #0FFH, AP_100

; Normal 2-digit entry required
MOV A, D2_ASC
CJNE A, #0FFH, AP2
RET
; only 1 digit entered -> ignore

```

AP2:

```

MOV A, D1_ASC
ANL A, #0FH
MOV B, #10
MUL AB
MOV R4, A
; tens*10

MOV A, D2_ASC
ANL A, #0FH
ADD A, R4
MOV R4, A
; 0..99
SJMP AP_DONE

```

AP_100:

```

; only exact "100" is valid as 3-digit entry
MOV A, D1_ASC
CJNE A, #'1', AP_INVALID
MOV A, D2_ASC
CJNE A, #'0', AP_INVALID
MOV A, D3_ASC
CJNE A, #'0', AP_INVALID
MOV A, #100
SJMP AP_DONE

```

AP_INVALID:

```

RET

```

AP_DONE:

```

MOV TARGET_DUTY, A
MOV D1_ASC, #0FFH
MOV D2_ASC, #0FFH
MOV D3_ASC, #0FFH

```

```
; While stopped: only update selected target shown on LCD.  
; While running: ramp from current duty to the new target in ACC
```

```
MOV A, RUN_STATE  
JZ AP_SETSTAT  
MOV A, DUTY  
CJNE A, TARGET_DUTY, AP_RAMP  
MOV ACC_FLAG, #00  
MOV RAMP_FLAG, #00  
MOV SHOW_ACC, #00  
SJMP AP_SETSTAT
```

AP_RAMP:

```
MOV ACC_FLAG, #01  
MOV ACC_TICKS, #00  
MOV RAMP_FLAG, #01  
MOV SHOW_ACC, #00  
MOV SHOW_MODE, #00
```

AP_SETSTAT:

```
ACALL UPDATE_RUN_AND_ACCEL  
MOV STATUS_REQ, #01  
RET
```

```
;=====
```

```
; DIGIT ENTRY
```

```
;=====
```

STORE_DIGIT:

```
MOV A, D1_ASC  
CJNE A, #0FFH, ST_D2  
MOV A, KEYBUF  
MOV D1_ASC, A  
RET
```

ST_D2:

```
MOV A, D2_ASC  
CJNE A, #0FFH, ST_D3  
MOV A, KEYBUF  
MOV D2_ASC, A  
RET
```

ST_D3:

```
MOV A, D3_ASC  
CJNE A, #0FFH, ST_EXIT
```

```
; Only allow third digit when first two digits are 1 and 0,  
; and the new digit is 0 => exact 100.
```

```
MOV A, D1_ASC  
CJNE A, #'1', ST_EXIT  
MOV A, D2_ASC  
CJNE A, #'0', ST_EXIT  
MOV A, KEYBUF  
CJNE A, #'0', ST_EXIT  
MOV D3_ASC, A
```

ST_EXIT:

```
RET
```

```

;=====
; RUN / ACCELERATION MANAGEMENT
; Maintains RUN_STATE. Startup begins at the currently selected
; target duty. ACC timing is used for in-run speed changes only.
;=====
UPDATE_RUN_AND_ACCEL:
    ; Determine run state = (DIR != 0) AND (TARGET_DUTY != 0)
    MOV A, DIR
    JZ UR_STOP
    MOV A, TARGET_DUTY
    JZ UR_STOP
    SJMP UR_RUN

UR_STOP:
    MOV A, RUN_STATE
    JZ UR_EXIT
    MOV RUN_STATE, #00
    MOV ACC_FLAG, #00
    MOV RAMP_FLAG, #00
    MOV ACC_TICKS, #00
    MOV SHOW_ACC, #00
    MOV SHOW_MODE, #00
    MOV FULL100, #00
    RET

UR_RUN:
    MOV A, RUN_STATE
    JNZ UR_EXIT

    ; New startup event after '+' or '-': start immediately at selected duty
    MOV RUN_STATE, #01
    MOV ACC_FLAG, #00
    MOV ACC_DONE, #00
    MOV ACC_TICKS, #00
    MOV FULL100, #00
    MOV TGT_TICKS, #00
    MOV MAX_TICKS, #00
    MOV SHOW_ACC, #00
    MOV SHOW_MODE, #00
    MOV RAMP_FLAG, #00
    MOV A, TARGET_DUTY
    MOV DUTY, A
    MOV STATUS_REQ, #01

UR_EXIT:
    RET

;=====
; PWM: 100-step software PWM on P3.7
; If DIR=0 or DUTY=0 => stop
;=====
PWM_ONE_PERIOD:
    MOV A, DIR
    JNZ PWM_CHK_DUTY
    CLR P3.7
    RET

```

```

PWM_CHK_DUTY:
    MOV A, DUTY

```

```
JNZ PWM_GO
CLR P3.7
RET
```

```
PWM_GO:
MOV R6, #100           ; steps 0..99
PWM_L:
MOV A, #100
CLR C
SUBB A, R6
MOV R5, A              ; step = 100-R6

; For duty = 100, keep PWM fully ON all cycle
MOV A, DUTY
CJNE A, #100, PWM_CMP
SETB P3.7
ACALL PWM_DELAY
DJNZ R6, PWM_L
RET
```

```
PWM_CMP:
MOV A, R5
CLR C
SUBB A, DUTY
JC PWM_ON
```

```
PWM_OFF:
CLR P3.7
ACALL PWM_DELAY
DJNZ R6, PWM_L
RET
```

```
PWM_ON:
SETB P3.7
ACALL PWM_DELAY
DJNZ R6, PWM_L
RET
```

```
PWM_DELAY:
MOV R7, #60
PD1: DJNZ R7, PD1
RET
```

```
;/=====
; LED INDICATION + FAULT DETECTION
;/=====
```

```
UPDATE_LEDS:
; clear previous display first
CLR LED1
CLR LED2
CLR LED3
MOV LED_FAULT_F, #00

; simulated LED fault for demo
MOV A, LED_FAULT_S
JZ UL_RUNCHK
MOV LED_FAULT_F, #01
RET
```

UL_RUNCHK:

```
; stopped or 0% speed => all LEDs off, no fault
MOV A,DIR
JZ UL_EXIT
MOV A,DUTY
JZ UL_EXIT

; 100% => blink all three together
MOV A,DUTY
CJNE A,#100,UL_RANGE
MOV A,BLINK_TGL
JZ UL_EXIT
SETB LED1
SETB LED2
SETB LED3
RET
```

UL_RANGE:

```
; 1..25 -> 1 LED
; 26..50 -> 2 LEDs
; 51..99 -> 3 LEDs
MOV A,DUTY
CLR C
SUBB A,#26
JC UL_LOW

MOV A,DUTY
CLR C
SUBB A,#51
JC UL_MID

; high range
SETB LED1
SETB LED2
SETB LED3
RET
```

UL_LOW:

```
SETB LED1
RET
```

UL_MID:

```
SETB LED1
SETB LED2
RET
```

UL_EXIT:

```
RET
```

```
;=====
; LCD STATUS DISPLAY
; Priority:
; OVERHEAT > LED FAULT > ACC RESULT > NORMAL STATUS
;=====
```

SERVICE_LCD:

```
MOV A,STATUS_REQ
JZ SV_RET
ACALL SHOW_STATUS
MOV STATUS_REQ,#00
```

SV_RET:

```
RET
```

SHOW_STATUS:

```
ACALL LCD_CLEAR

MOV A, OVH_FLAG
JZ SH_LEDCHK
MOV DPTR, #MSG_OVH1
ACALL LCD_PUTS
ACALL LCD_LINE2
MOV DPTR, #MSG_OVH2
ACALL LCD_PUTS
RET
```

SH_LEDCHK:

```
MOV A, LED_FAULT_F
JZ SH_NORMAL
MOV DPTR, #MSG_LF1
ACALL LCD_PUTS
ACALL LCD_LINE2
MOV DPTR, #MSG_LF2
ACALL LCD_PUTS
RET
```

SH_NORMAL:

```
MOV DPTR, #MSG_DIR
ACALL LCD_PUTS

MOV A, DIR
CJNE A, #01, SH_CCW
MOV DPTR, #MSG_CW
ACALL LCD_PUTS
SJMP SH_L2
```

SH_CCW:

```
CJNE A, #02, SH_STOP
MOV DPTR, #MSG_CCW
ACALL LCD_PUTS
SJMP SH_L2
```

SH_STOP:

```
MOV DPTR, #MSG_STOP
ACALL LCD_PUTS
```

SH_L2:

```
ACALL LCD_LINE2

; show ACC result temporarily if available
MOV A, SHOW_ACC
JZ SH_SPD
MOV A, SHOW_MODE
CJNE A, #01, SH_MAXMSG
MOV DPTR, #MSG_ACC
ACALL LCD_PUTS
MOV A, TGT_TICKS
ACALL LCD_SHOW_TICKS
RET
```

SH_MAXMSG:

SH_SPD:

```
MOV DPTR, #MSG_SPD
ACALL LCD_PUTS
```

```
; If user is entering digits, show input buffer
```

```
MOV A,D1_ASC
```

```
CJNE A,#0FFH,SH_D1
```

```
SJMP SH_SHOW_DUTY
```

```
SH_D1:
```

```
MOV A,D1_ASC
```

```
ACALL LCD_DATA
```

```
MOV A,D2_ASC
```

```
CJNE A,#0FFH,SH_D2
```

```
MOV A,'#_'
```

```
ACALL LCD_DATA
```

```
MOV A,'%'
```

```
ACALL LCD_DATA
```

```
RET
```

```
SH_D2:
```

```
MOV A,D2_ASC
```

```
ACALL LCD_DATA
```

```
MOV A,D3_ASC
```

```
CJNE A,#0FFH,SH_D3
```

```
MOV A,'%'
```

```
ACALL LCD_DATA
```

```
RET
```

```
SH_D3:
```

```
MOV A,D3_ASC
```

```
ACALL LCD_DATA
```

```
MOV A,'%'
```

```
ACALL LCD_DATA
```

```
RET
```

```
SH_SHOW_DUTY:
```

```
ACALL LCD_SHOW_DUTY
```

```
RET
```

```
LCD_SHOW_DUTY:
```

```
MOV A,TARGET_DUTY
```

```
CJNE A,#100,LSD2
```

```
MOV A,'#1'
```

```
ACALL LCD_DATA
```

```
MOV A,'#0'
```

```
ACALL LCD_DATA
```

```
MOV A,'#0'
```

```
ACALL LCD_DATA
```

```
MOV A,'%'
```

```
ACALL LCD_DATA
```

```
RET
```

```
LSD2:
```

```
MOV B,#10
```

```
DIV AB
```

```
; A=tens, B=ones
```

```
ADD A,'#0'
```

```
ACALL LCD_DATA
```

```
MOV A,B
```

```
ADD A,'#0'
```

```

ACALL LCD_DATA
MOV A, #'%'
ACALL LCD_DATA
RET

```

; Show ticks already loaded in A as xx.xS from 100 ms units

LCD_SHOW_TICKS:

```

MOV B, #10
DIV AB                ; A=seconds (0..25), B=tenths
MOV R4, A             ; seconds
MOV R5, B             ; tenths

MOV A, R4
MOV B, #10
DIV AB                ; A=tens of seconds, B=ones of seconds
ADD A, #'0'
ACALL LCD_DATA
MOV A, B
ADD A, #'0'
ACALL LCD_DATA
MOV A, #'.'
ACALL LCD_DATA
MOV A, R5
ADD A, #'0'
ACALL LCD_DATA
MOV A, #'S'
ACALL LCD_DATA
RET

```

```

;=====
; Keypad: full scan + map (calculator keypad)
;=====

```

GETKEY:

```

MOV P1, #0FEH        ; Row A
ACALL READCOL
JZ  ROWB
ACALL MAP_ROWA
RET

```

ROWB:

```

MOV P1, #0FDH        ; Row B
ACALL READCOL
JZ  ROWC
ACALL MAP_ROWB
RET

```

ROWC:

```

MOV P1, #0FBH        ; Row C
ACALL READCOL
JZ  ROWD
ACALL MAP_ROWC
RET

```

ROWD:

```

MOV P1, #0F7H        ; Row D
ACALL READCOL
JZ  NONE
ACALL MAP_ROWD
RET

```

NONE:

```
MOV A, #00H
MOV P1, #0FFH
RET
```

; A=1..4 column

MAP_ROW A:

```
CJNE A, #1, MA2
MOV A, #'7'
SJMP MEND
```

MA2: CJNE A, #2, MA3

```
MOV A, #'8'
SJMP MEND
```

MA3: CJNE A, #3, MA4

```
MOV A, #'9'
SJMP MEND
```

MA4:

```
MOV A, #'/' ; STOP key ÷
```

MEND:

```
MOV P1, #0FFH
RET
```

MAP_ROW B:

```
CJNE A, #1, MB2
MOV A, #'4'
SJMP MEND2
```

MB2: CJNE A, #2, MB3

```
MOV A, #'5'
SJMP MEND2
```

MB3: CJNE A, #3, MB4

```
MOV A, #'6'
SJMP MEND2
```

MB4:

```
MOV A, #'*' ; LED fault demo toggle
```

MEND2:

```
MOV P1, #0FFH
RET
```

MAP_ROW C:

```
CJNE A, #1, MC2
MOV A, #'1'
SJMP MEND3
```

MC2: CJNE A, #2, MC3

```
MOV A, #'2'
SJMP MEND3
```

MC3: CJNE A, #3, MC4

```
MOV A, #'3'
SJMP MEND3
```

MC4:

```
MOV A, #'-'
```

MEND3:

```
MOV P1, #0FFH
RET
```

MAP_ROW D:

```
CJNE A, #1, MD2
MOV A, #'C'
SJMP MEND4
```

```

MD2:CJNE A,#2,MD3
    MOV A,'#0'
    SJMP MEND4
MD3:CJNE A,#3,MD4
    MOV A,'#='
    SJMP MEND4
MD4:
    MOV A,'#+'
MEND4:
    MOV P1,#0FFH
    RET

```

```

;-----
; READCOL: A=1..4 if any column low, else 0
;-----
READCOL:
    ACALL DLY_1MS
    JB P1.4,RC2
    MOV A,#01
    RET
RC2:
    JB P1.5,RC3
    MOV A,#02
    RET
RC3:
    JB P1.6,RC4
    MOV A,#03
    RET
RC4:
    JB P1.7,RNONE
    MOV A,#04
    RET
RNONE:
    MOV A,#00
    RET

DLY_1MS:
    MOV R6,#5
DMS1:
    MOV R7,#250
DMS2:DJNZ R7,DMS2
    DJNZ R6,DMS1
    RET

DELAY_100MS:
    MOV R6,#100
D100_1:
    ACALL DLY_1MS
    DJNZ R6,D100_1
    RET

;=====
; LCD routines (8-bit)
;=====
LCD_PWRDELAY:
    ACALL LCD_LONG
    ACALL LCD_LONG
    RET

```

```
LCD_INIT:
    MOV A, #038H
    ACALL LCD_CMD
    MOV A, #00CH
    ACALL LCD_CMD
    MOV A, #006H
    ACALL LCD_CMD
    RET
```

```
LCD_CLEAR:
    MOV A, #001H
    ACALL LCD_CMD
    ACALL LCD_LONG
    RET
```

```
LCD_LINE2:
    MOV A, #0C0H
    ACALL LCD_CMD
    RET
```

```
LCD_CMD:
    CLR LCD_RS
    MOV P2, A
    SETB LCD_E
    ACALL LCD_SHORT
    CLR LCD_E
    ACALL LCD_SHORT
    RET
```

```
LCD_DATA:
    SETB LCD_RS
    MOV P2, A
    SETB LCD_E
    ACALL LCD_SHORT
    CLR LCD_E
    ACALL LCD_SHORT
    RET
```

```
LCD_PUTS:
LP1: CLR A
    MOVC A, @A+DPTR
    JZ LP2
    ACALL LCD_DATA
    INC DPTR
    SJMP LP1
LP2: RET
```

```
LCD_SHORT:
    MOV R7, #80
LS1: DJNZ R7, LS1
    RET
```

```
LCD_LONG:
    MOV R6, #200
LL1: MOV R7, #250
LL2: DJNZ R7, LL2
    DJNZ R6, LL1
    RET
```

```
;----- Strings -----  
MSG_DIR:  DB 'DIR:',0  
MSG_CW:   DB 'CW',0  
MSG_CCW:  DB 'CCW',0  
MSG_STOP: DB 'STOP',0  
MSG_SPD:  DB 'SPD:',0  
MSG_ACC:  DB 'ACC:',0  
MSG_MAX:  DB 'MAX:',0  
MSG_OVH1: DB 'OVERHEAT',0  
MSG_OVH2: DB 'SAFE:50 C=RST',0  
MSG_LF1:  DB 'LED FAULT',0  
MSG_LF2:  DB 'CHECK/CLR(*)',0  
  
END
```

7. Hardware Implementation

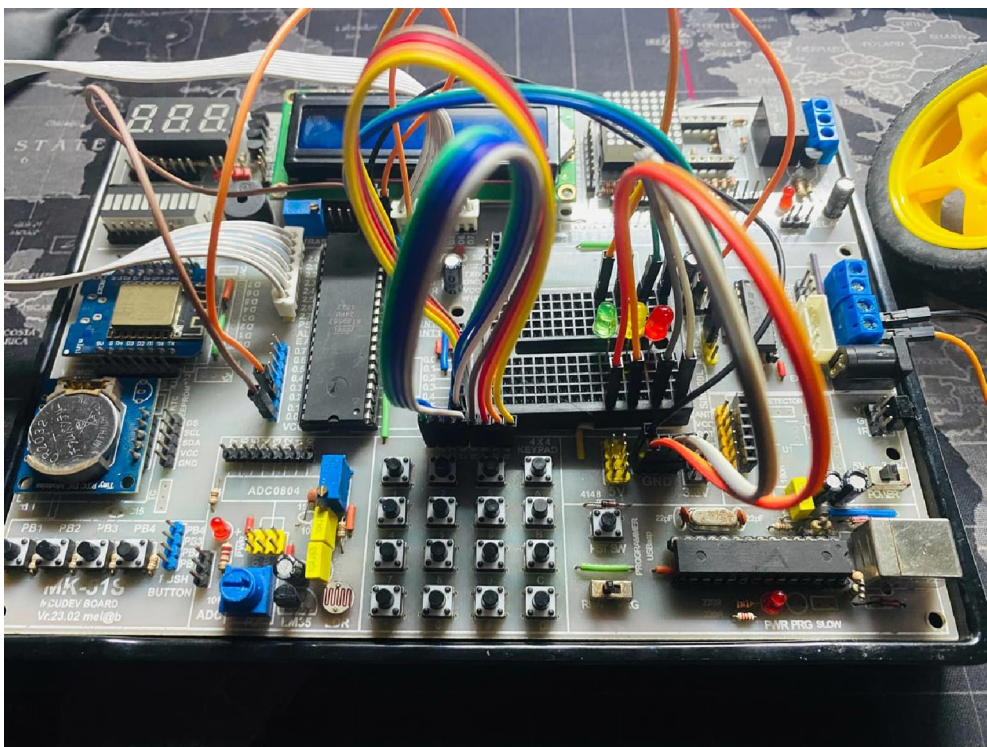
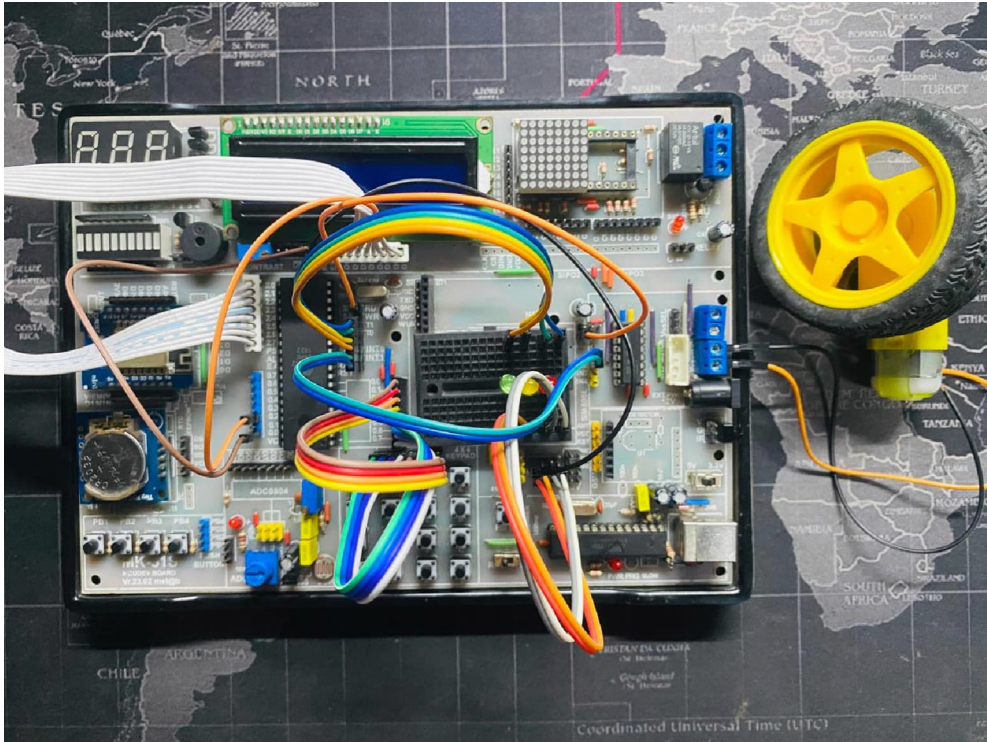


Figure 8. Hardware demonstration

8. Problems Faced and Solutions

- **Address out of range in MIDE-51:** Short conditional jumps such as JZ and SJMP exceeded their reach in some branches. The fix was to restructure the logic with nearby labels and use LJMP for long transfers.
- **Soft reverse initially stuck at STOP in Proteus:** The first versions attempted reversal while the opposite-direction key was still being detected. The routine was corrected to wait for key release explicitly, then apply the stop delay, and only then issue the new direction command.
- **Requirement mismatch between “2-digit speed” and “100% overheat test”:** A two-digit parser cannot represent 100. This was solved by keeping normal two-digit entry for 00–99 and adding one exact 100 path using the sequence 1–0–0–=.
- **Interrupt Register Conflict and Register-Bank Isolation:** Registers used inside the interrupt service routine initially risked corrupting foreground code. The ISR was moved to register bank 1 so that R4 and related working registers in the main program remain safe.
- **Speed-Apply Logic Bug and Ramp-Control Correction:** The first version of the speed-apply routine wrote to DUTY immediately during normal two-digit entry, which prevented the desired-speed timing feature from operating correctly. This problem was corrected by updating only TARGET_DUTY and allowing the interrupt service routine to gradually ramp DUTY until the target value was reached.
- **LED fault detection could not be sensed electrically with the available circuit:** A simulated fault mode was added through the × key so the feature could still be demonstrated and explained within the existing hardware constraints.
- **Hardware implementation and LCD display debugging:** During practical hardware implementation, the LCD initially did not display characters properly because of wiring and interfacing issues during setup. The problem was debugged by checking the power and ground connections, verifying the LCD pin mapping, correcting the data/control line connections, adjusting the contrast setting, and retesting the initialization sequence until the display worked reliably.

9. Conclusion

This project successfully demonstrates a complete 8051-based DC motor controller with both functional and protective features. The final system accepts keypad commands, controls direction and speed through software PWM, handles safe reversal, displays status on an LCD, and enforces a simulated thermal cutoff after prolonged full-speed operation. In addition, the report documents two useful extensions: acceleration-time monitoring and LED-based speed visualization with fault demonstration.

From an educational perspective, the project integrates digital interfacing, time-based firmware design, state management, and safety logic in a single embedded application. The Proteus implementation provides a strong foundation for later migration to a physical prototype, where the same logic can be extended further with real RPM sensing or temperature feedback if required.